

# Creational pattern

---

A **creational pattern** is a software design pattern for creating objects in a manner suitable to a particular situation. As object creation that is otherwise available (i.e. via the programming language) can sometimes result in design limitations, a custom mechanism for creation can provide for better design. A creational pattern aims to separate a system from how its objects are created, composed, and represented. They increase the system's flexibility in terms of the what, who, how, and when of object creation.<sup>[1]</sup>

A creational pattern encapsulates two main aspects. One is encapsulating knowledge about which concrete classes the system uses. Another is hiding how instances of these concrete classes are created and combined.<sup>[2]</sup>

Creational design patterns are categorized into object-creational patterns and class-creational patterns. Object-creational patterns defer part of its object creation to another object, while class-creational patterns defer its object creation to subclasses.<sup>[3]</sup>

## Usage

---

As modern software engineering depends more on object composition than class inheritance, emphasis shifts away from hard-coding behaviors toward defining a smaller set of basic behaviors that can be composed into more complex ones.<sup>[4]</sup> Hard-coding behaviors are inflexible because they require overriding or re-implementing the whole thing in order to change parts of the design. Additionally, hard-coding does not promote reuse and makes it difficult to keep track of errors. For these reasons, creational patterns are more useful than hard-coding behaviors. Creational patterns make design become more flexible. They provide different ways to remove explicit references in the concrete classes from the code that needs to instantiate them.<sup>[5]</sup> In other words, they create independency for objects and classes.

Consider applying creational patterns when:

- A system should be independent of how its objects and products are created.
- A set of related objects is designed to be used together.
- Hiding the implementations of a class library or product, revealing only their interfaces.
- Constructing different representation of independent complex objects.
- A class wants its subclass to implement the object it creates.
- The class instantiations are specified at run-time.
- There must be a single instance and client can access this instance at all times.
- Instance should be extensible without being modified.

## Structure

---

Below is a simple class diagram that most creational patterns have in common. Note that different creational patterns require additional and different participated classes.

## Participants:

- **Creator:** Declares object interface. Returns object.
- **ConcreteCreator:** Implements object's interface.

## Examples

---

Some examples of creational design patterns include:

### Abstract factory pattern

a class requests the objects it requires from a factory object instead of creating the objects directly.<sup>[6]</sup>

### Factory method pattern

centralize creation of an object of a specific type choosing one of several implementations.<sup>[7]</sup>

### Builder pattern

separate the construction of a complex object from its representation so that the same construction process can create different representations.

### Dependency injection pattern

a class accepts the objects it requires from an injector instead of creating the objects directly

### Lazy initialization pattern

tactic of delaying the creation of an object, the calculation of a value, or some other expensive process until the first time it is needed.

### Object pool pattern

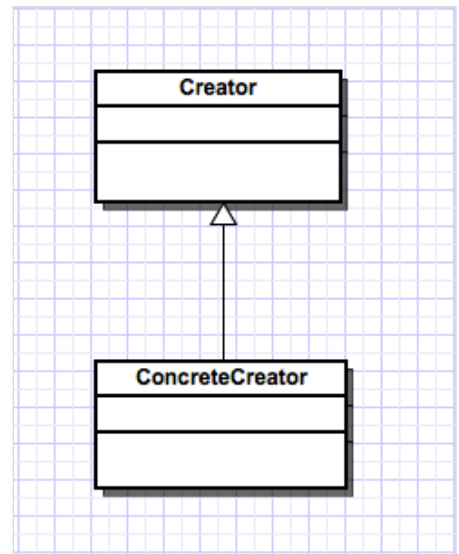
avoid expensive acquisition and release of resources by recycling objects that are no longer in use.

### Prototype pattern

used when the type of objects to create is determined by a prototypical instance, which is cloned to produce new objects.

### Singleton pattern

restrict instantiation of a class to one object.<sup>[8]</sup>



Creational Pattern class diagram. 

## See also

---

- Constructor
- Behavioral pattern
- Concurrency pattern
- Structural pattern

## References

---

1. Judith, Bishop (2007). *C# 3.0 Design Patterns* (<https://archive.org/details/c30designpattern000bish/page/336>). California: O'Reilly Media. p. 336 (<https://archive.org/details/c30designpattern0000bish/page/336>). ISBN 978-0-596-52773-0. Retrieved 2015-05-22.
2. Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John (1995). *Design Patterns* (<https://archive.org/details/designpatternsel00gamm/page/81>). Massachusetts: Addison-Wesley. p. 81 (<https://archive.org/details/designpatternsel00gamm/page/81>). ISBN 978-0-201-63361-0. Retrieved 2015-05-22.

3. Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John (1995). *Design Patterns* (<https://archive.org/details/designpatternsel00gamm>). Massachusetts: Addison-Wesley. ISBN 978-0-201-63361-0. Retrieved 2015-05-22.
4. Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John (1995). *Design Patterns* (<https://archive.org/details/designpatternsel00gamm/page/84>). Massachusetts: Addison-Wesley. p. 84 (<https://archive.org/details/designpatternsel00gamm/page/84>). ISBN 978-0-201-63361-0. Retrieved 2015-05-22.
5. Gamma, Erich; Helm, Richard; Johnson, Ralph; Vlissides, John (1995). *Design Patterns* (<https://archive.org/details/designpatternsel00gamm/page/85>). Massachusetts: Addison-Wesley. p. 85 (<https://archive.org/details/designpatternsel00gamm/page/85>). ISBN 978-0-201-63361-0. Retrieved 2015-05-22.
6. Freeman, Eric; Freeman, Elisabeth; Sierra, Kathy; Bates, Bert (2004). Hendrickson, Mike; Loukides, Mike (eds.). *Head First Design Patterns* (<http://shop.oreilly.com/product/9780596007126.do>). California: O'Reilly Media. p. 156. ISBN 978-0-596-00712-6. Retrieved 2015-05-22.
7. Freeman, Eric; Freeman, Elisabeth; Sierra, Kathy; Bates, Bert (2004). Hendrickson, Mike; Loukides, Mike (eds.). *Head First Design Patterns* (<http://shop.oreilly.com/product/9780596007126.do>). California: O'Reilly Media. p. 134. ISBN 978-0-596-00712-6. Retrieved 2015-05-22.
8. Freeman, Eric; Freeman, Elisabeth; Sierra, Kathy; Bates, Bert (2004). Hendrickson, Mike; Loukides, Mike (eds.). *Head First Design Patterns* (<http://shop.oreilly.com/product/9780596007126.do>). California: O'Reilly Media. p. 177. ISBN 978-0-596-00712-6. Retrieved 2015-05-22.

---

Retrieved from "[https://en.wikipedia.org/w/index.php?title=Creational\\_pattern&oldid=1327663100](https://en.wikipedia.org/w/index.php?title=Creational_pattern&oldid=1327663100)"